

Working of CoAP and MQTT Protocols in IoT

Detailed Working Principles, Worked IoT Examples, Advantages & Disadvantages

Presented by: Dr. Anand D. Mane

Session Objectives

- Understand why HTTP is unsuitable for constrained IoT devices, motivating CoAP and MQTT.
- Study the internal architecture and message-level working of CoAP, step by step.
- Study the internal architecture and message-level working of MQTT, step by step.
- Trace two complete, real IoT scenarios for each protocol, message by message.
- Compare advantages and disadvantages of CoAP and MQTT for different deployment needs.

Recap — Why Not Plain HTTP for Constrained Devices?

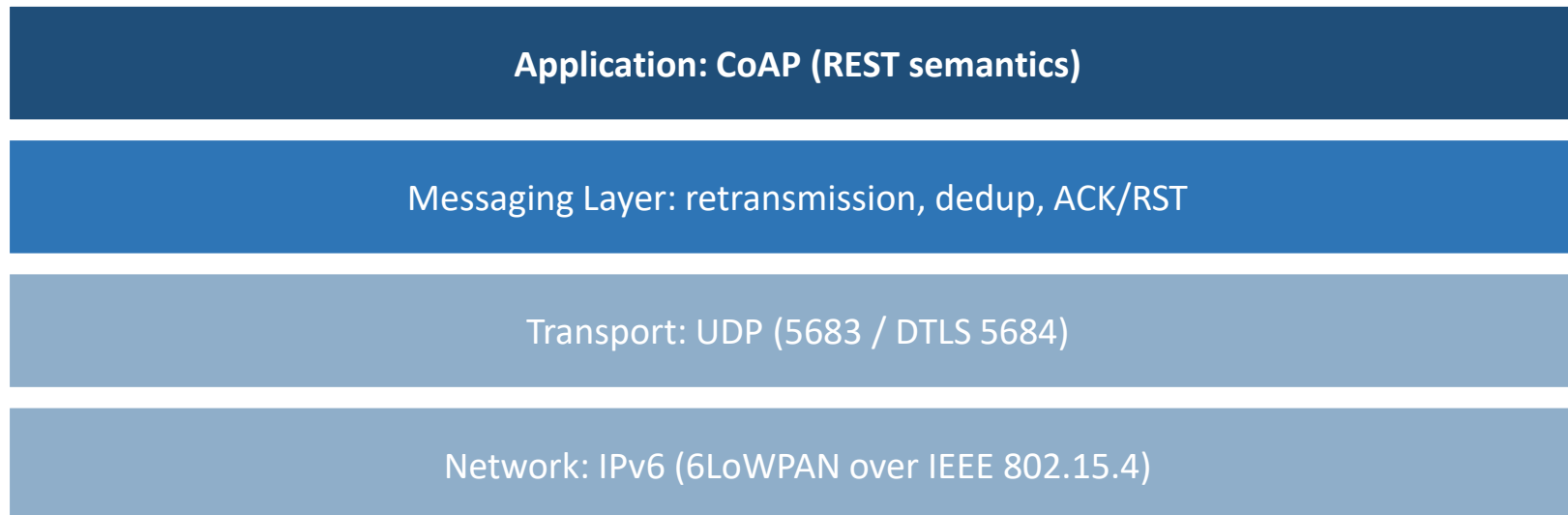
- HTTP runs over TCP: every request pays for a 3-way handshake (and a TLS handshake for HTTPS) before any data moves.
- HTTP headers are verbose ASCII text — hundreds of bytes just to move a few bytes of sensor data.
- Constrained nodes (battery-powered, kilobytes of RAM, low-power radios) cannot afford this overhead on every message.
- CoAP and MQTT were both designed specifically to reduce this overhead — but using two very different strategies, which we study in this session.

Part A

CoAP — Constrained Application Protocol

2.1 CoAP — Architecture Overview

- Runs over UDP — port 5683 (plain) or 5684 for DTLS-secured CoAP — connectionless and lightweight.
- Two internal layers: a Messaging layer (retransmission, deduplication) and a Request/Response layer (RESTful GET/POST/PUT/DELETE semantics).
- Every message begins with a compact 4-byte binary header, followed by an optional Token, compact Options, and an optional Payload.



2.1 CoAP — Message Header and Message Types

- CON (Confirmable): needs acknowledgement — retransmitted with exponential backoff until ACK or retries exhausted.
- NON (Non-confirmable): fire-and-forget — sent once, no retransmission (e.g., periodic telemetry).
- ACK (Acknowledgement): confirms a CON message, often carries the response piggybacked.
- RST (Reset): 'I received this but cannot process it' — sent for unknown or invalid requests.

2.2 CoAP — General Working (1/2)

Step 1: The client (gateway/controller) builds a CoAP request with a Method Code (GET/POST/PUT/DELETE), a Message ID, a Token, and a Uri-Path option naming the target resource.

Step 2: The client chooses CON (guaranteed delivery) or NON (occasional loss acceptable, e.g. periodic sensor telemetry).

Step 3: The request is sent as a single UDP datagram to the constrained server (sensor/actuator node).

Step 4: If CON and no ACK arrives within ACK_TIMEOUT (~2s, randomised), the client retransmits the identical datagram with the same Message ID, doubling the timeout each retry (exponential backoff), up to MAX_RETRANSMIT (default 4).

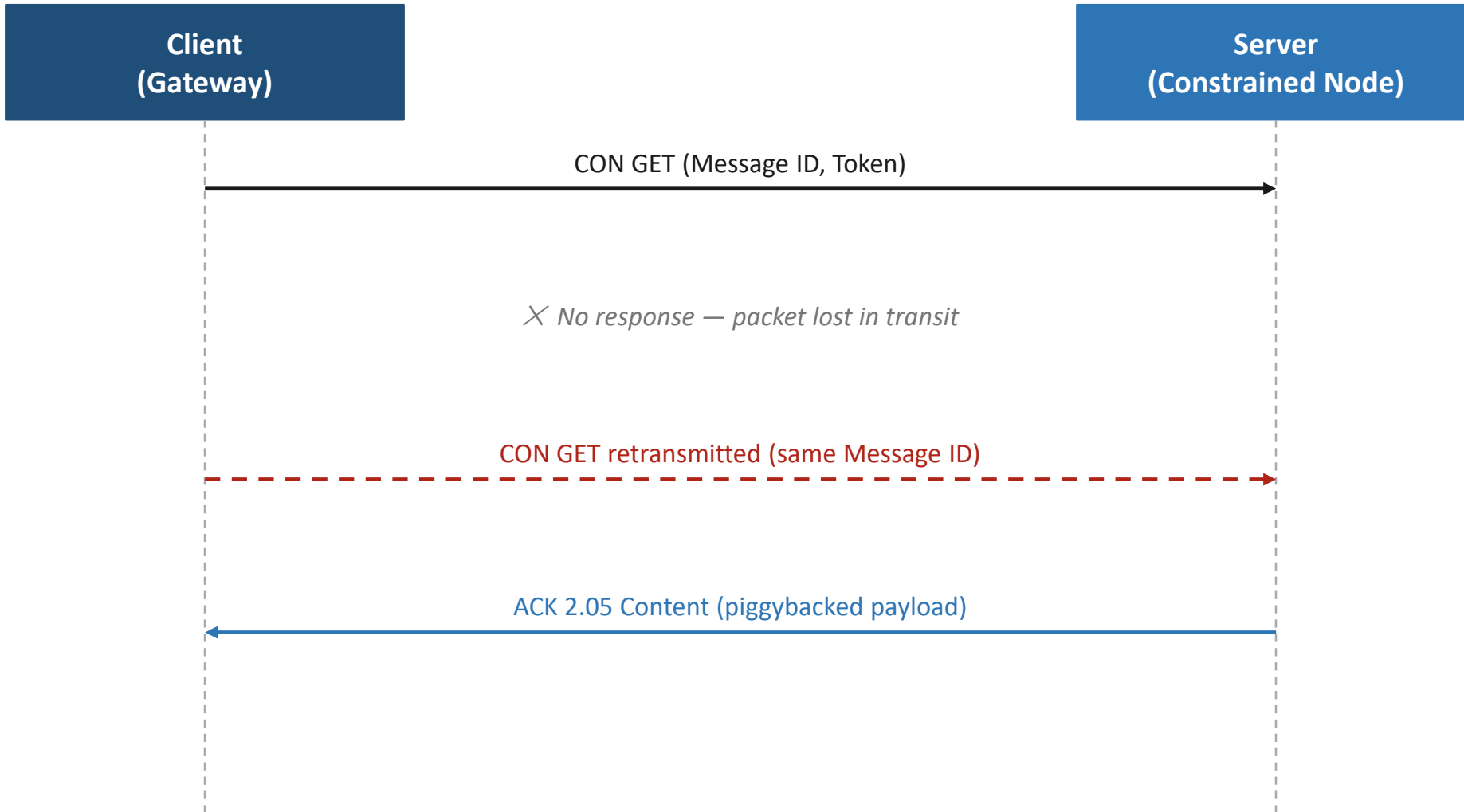
2.2 CoAP — General Working (2/2)

Step 5: The server processes the request and replies — usually piggybacked on the ACK; if processing is slow, an empty ACK is sent first, followed later by a separate CON response the client must itself ACK.

Step 6: The client's deduplication cache matches the Message ID and silently discards duplicate ACKs/responses caused by retransmission.

Step 7 (optional): For continuous updates, the client registers interest using the Observe option instead of polling; the server then pushes a new notification whenever the observed resource changes.

2.2 CoAP — Generic Confirmable Request/Response Flow



After ACK_TIMEOUT, backoff doubles on each retry, up to MAX_RETRANSMIT retries.

2.3 Worked Example 1 — Smart Street Light Brightness Read

- **Scenario**

- A city traffic-management gateway queries a solar-powered street-light controller for its current brightness/dimming level, over a lossy LoRaWAN-to-IP bridge.

- **Why CoAP here?**

- The controller is battery/solar powered with a low-power radio link — CoAP's small header and UDP transport avoid the cost of a TCP handshake on every reading, while CON still guarantees the reading is not silently lost.

2.3 Example 1 — Working

Step 1: Gateway sends a CON GET request:

```
CoAP CON GET /streetlight/12/brightness MID=0x4B10 Token=0x2C
```

Step 2: The datagram is lost due to interference on the wireless bridge — no ACK is received.

Step 3: After ACK_TIMEOUT (~2s), the gateway retransmits the identical CON GET with the same Message ID (0x4B10).

2.3 Example 1 — Working (contd.)

Step 4: The retransmitted datagram reaches the controller, which reads its dimmer setting and replies immediately, piggybacked on the ACK:

```
CoAP ACK 2.05 Content MID=0x4B10 Token=0x2C  
Content-Format: application/json  
Payload: {"brightness": 65}
```

Step 5: The gateway matches the ACK to its request via Message ID and Token, updates the dashboard, and discards any late duplicate ACK.

Outcome: One retry recovers the reading despite a lost packet — cheaper than a TCP-based retry over the same lossy link.

2.4 Worked Example 2 — Smart Irrigation Soil-Moisture (Observe)

- **Scenario**

- A farm irrigation controller needs continuously updated soil-moisture readings from a buried sensor node, without constantly polling it (which would drain the node's battery).
- **Why CoAP Observe here?**
- Polling wastes energy on a battery-buried node. The Observe option lets the sensor push updates only when the value actually changes — pub/sub-like behaviour without needing a broker.

2.4 Example 2 — Working

Step 1: Controller sends a GET with the Observe option set to 0:

```
CoAP CON GET /field3/soil-moisture Observe=0 MID=0x9F21 Token=0x77
```

Step 2: The sensor registers the controller as an observer and returns the current reading with Observe sequence number 1:

```
CoAP ACK 2.05 Content Observe=1 MID=0x9F21 Token=0x77  
Payload: {"moisture": 38}
```

Step 3: When the value later changes meaningfully (e.g., after irrigation), the sensor proactively pushes a new notification with Observe=2 — without waiting for another request.

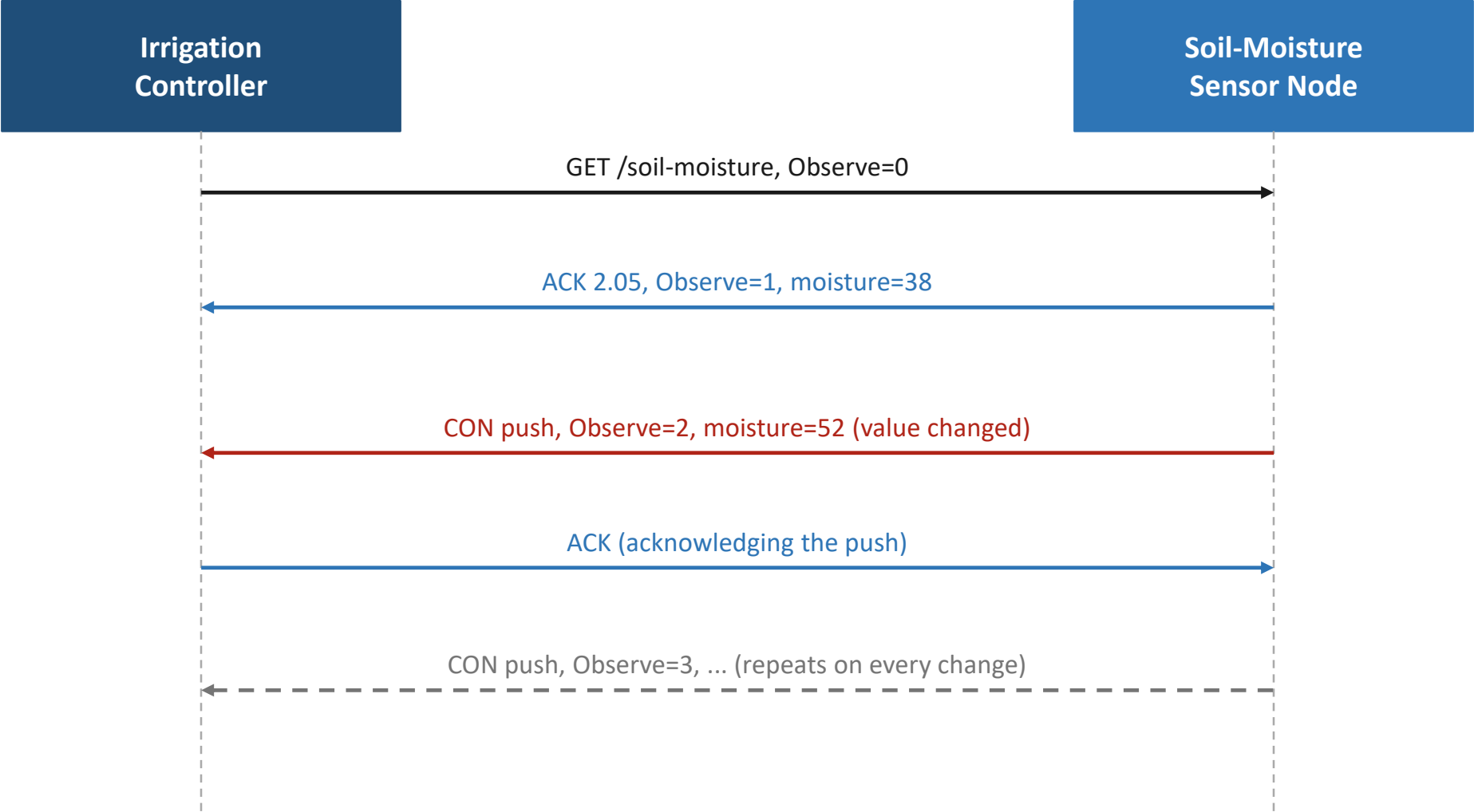
2.4 Example 2 — Working (contd.)

Step 4: The controller ACKs each pushed CON notification, keeping the observation relationship alive.

Step 5: To stop updates, the controller sends a GET with Observe removed, or simply lets the registration expire.

Outcome: The controller receives moisture updates only when values change, conserving the buried sensor's battery — pub/sub-like behaviour with only two parties, no broker.

2.4 CoAP Observe — Sequence Diagram



2.5 CoAP — Advantages (1/2)

Advantages

Very low overhead (4-20 bytes) — ideal for battery-powered, constrained nodes.

Runs over UDP, avoiding the cost of TCP connection setup on lossy links.

RESTful design maps naturally to HTTP, easing gateway translation to the web.

2.5 CoAP — Advantages (2/2)

Advantages

Built-in, tunable reliability (CON/NON) lets the application choose when to pay for retries.

Observe option provides lightweight push updates without needing a broker.

2.5 CoAP — Disadvantages (1/2)

Disadvantages

UDP is connectionless, so CoAP must reimplement reliability itself (retransmission, deduplication).

No native multi-subscriber fan-out; remains a two-party protocol, unlike MQTT's broker model.

Security (DTLS) has more implementation complexity than TLS-over-TCP in some constrained stacks.

2.5 CoAP — Disadvantages (2/2)

Disadvantages

NAT traversal for UDP can be harder than for TCP in some network setups.

Less mature tooling and ecosystem support compared with MQTT in commercial IoT platforms.

Part B

MQTT — Message Queuing Telemetry Transport

3.1 MQTT — Architecture Overview

- Runs over TCP — port 1883 (plain) or 8883 for MQTTS/TLS — and follows a publish/subscribe model mediated by a central Broker.
- Publishers and Subscribers never talk directly; both connect only to the broker, fully decoupling them in time, space, and synchronisation.



3.1 MQTT — Topics, QoS, and Extra Features

- Messages are published to hierarchical, slash-separated Topics (e.g., factory1/line2/motor7/vibration); subscribers use Topic Filters with + (single-level) and # (multi-level) wildcards.
- Three Quality of Service levels chosen per message: QoS 0 (at most once), QoS 1 (at least once), QoS 2 (exactly once) — detailed on the next slide.
- Retained messages (broker stores the last value for new subscribers) and Last Will and Testament, LWT (a message the broker auto-publishes if a client disconnects ungracefully).

3.1 MQTT — Quality of Service (QoS) Levels

QoS	Guarantee	Handshake
0 — At most once	Fire-and-forget; message may be lost	PUBLISH only, no acknowledgement
1 — At least once	Delivered, possibly duplicated	PUBLISH -> PUBACK
2 — Exactly once	Delivered exactly once, no duplicates	PUBLISH -> PUBREC -> PUBREL -> PUBCOMP

3.2 MQTT — General Working (1/2)

Step 1: Publisher and subscriber each open a TCP connection to the broker and send CONNECT (Client ID, Keep-Alive interval, optional credentials, optional Last Will).

Step 2: The broker validates the connection and replies with CONNACK (return code 0 = accepted) to each client.

Step 3: Subscribers send SUBSCRIBE with one or more topic filters and a requested QoS; the broker replies with SUBACK confirming the granted QoS.

Step 4: The publisher sends PUBLISH with the topic name, payload, and chosen QoS.

3.2 MQTT — General Working (2/2)

Step 5: Depending on QoS, broker and publisher run the matching handshake (none for QoS 0; PUBACK for QoS 1; PUBREC/PUBREL/PUBCOMP for QoS 2) to confirm delivery to the broker.

Step 6: The broker independently forwards the message as its own PUBLISH to every currently matching subscriber, running the same QoS handshake with each.

Step 7: If a client is silent for more than $1.5 \times$ its Keep-Alive interval, the broker treats it as disconnected and publishes its Last Will message.

Step 8: If a publisher marks a message Retain=true, the broker stores it and immediately delivers it to any client that subscribes to that topic later.

3.3 Worked Example 1 — Smart Home Temperature Monitoring

- **Scenario**

- A temperature sensor in a smart-home living room publishes readings; a mobile-app dashboard subscribes to display them in real time.

- **Why MQTT at QoS 1 here?**

- The dashboard needs the reading reliably, but an occasional harmless duplicate is fine — QoS 1 ('at least once') gives that balance at lower cost than QoS 2.

3.3 Example 1 — Working

Steps 1-2: Sensor and mobile app both connect; broker replies CONNACK (accepted) to both.

```
CONNECT ClientId=temp-sensor-1r KeepAlive=60s
```

```
CONNECT ClientId=mobile-app-1 KeepAlive=60s
```

Step 3: The mobile app subscribes to the topic:

```
SUBSCRIBE PacketId=1 Topic=home/livingroom/temperature QoS=1
```

```
SUBACK PacketId=1 GrantedQoS=1
```

Step 4: The sensor publishes a new reading at QoS 1:

```
PUBLISH PacketId=45 Topic=home/livingroom/temperature Payload={"tempC":23.1} QoS=1
```

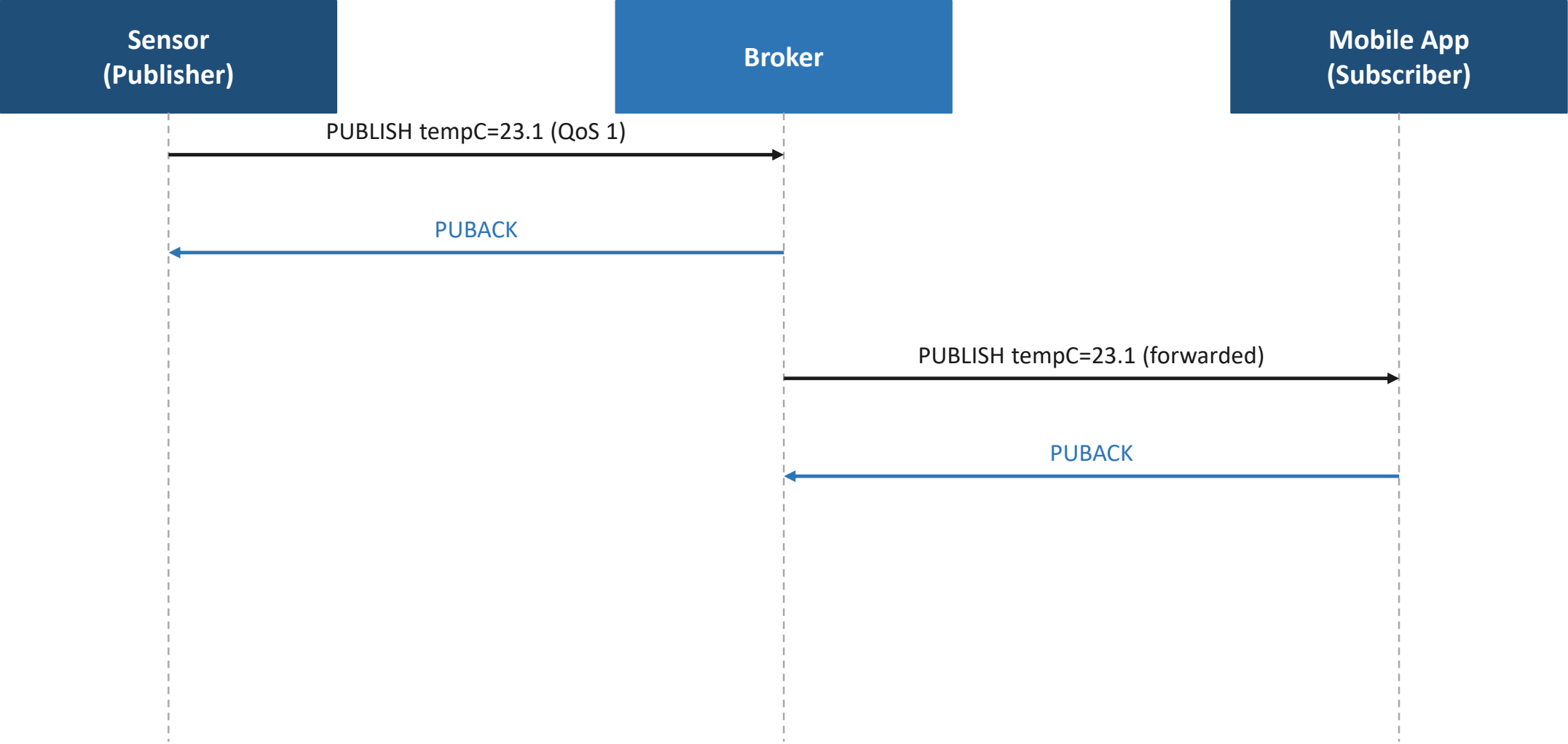
3.3 Example 1 — Delivery and Outcome

Step 5: The broker stores/forwards the message and returns PUBACK to the sensor, confirming receipt at the broker.

Step 6: The broker forwards the same reading to the mobile app as its own PUBLISH; the app responds with its own PUBACK.

Outcome: The dashboard reflects the room temperature within milliseconds; 'at least once' delivery ensures the reading is not silently lost, at the cost of possible harmless duplicates.

3.3 Example 1 — QoS 1 Sequence Diagram



3.4 Worked Example 2 — Industrial Machine-Health Monitoring

- **Scenario**
- A factory floor has 50 vibration sensors on production motors. The monitoring service must know each motor's exact current vibration state, be alerted instantly if a sensor drops offline, and never miscount a critical vibration spike.
- **Why MQTT at QoS 2 + Retain + Last Will here?**
- A missed or duplicated spike alert could mean a missed or false machine-failure warning — QoS 2 guarantees exactly-once delivery; Retain and Last Will add current-state and liveness monitoring for free.

3.4 Example 2 — Working (1/3)

Step 1: Each vibration sensor connects with a Last Will registered for its own status topic:

```
CONNECT ClientId=motor7-vib KeepAlive=30s  
  Will Topic=factory1/line2/motor7/status  
  Will Message="offline" Will QoS=1 Retain=true
```

Step 2: The monitoring service subscribes to all motors on line 2 using a multi-level wildcard:

```
SUBSCRIBE Topic=factory1/line2/# QoS=2  
SUBACK GrantedQoS=2
```


3.4 Example 2 — Working (2/3)

Step 3: Motor 7 detects a vibration spike above threshold and publishes at QoS 2, the highest guarantee, so the alert is neither lost nor duplicated:

```
PUBLISH Topic=factory1/line2/motor7/vibration  
  Payload={"level":8.7,"unit":"mm/s"}  QoS=2  Retain=true
```

QoS 2 is chosen here because a missed or duplicated spike alert could mean a missed or false machine-failure warning — the highest guarantee is worth the extra handshake cost.

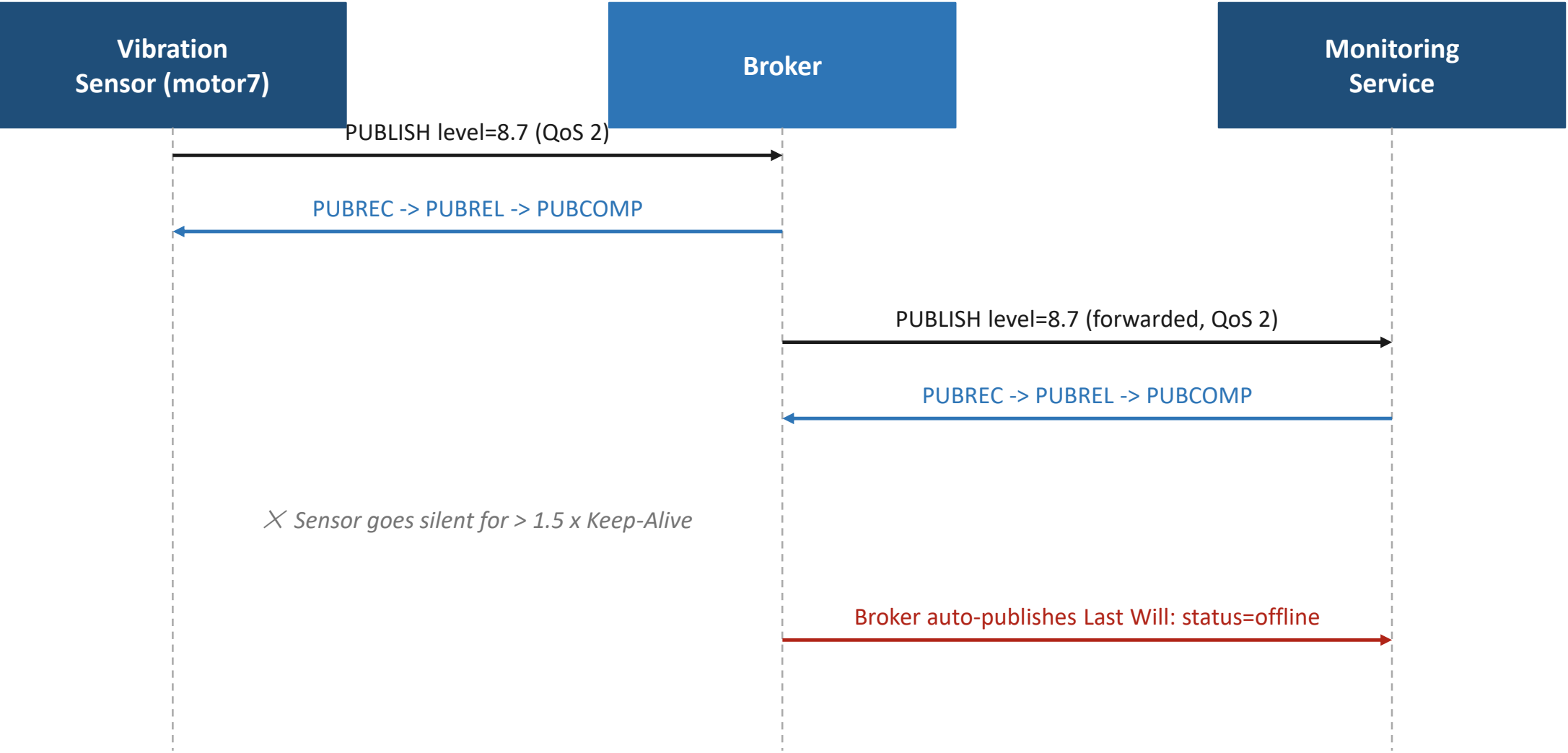
3.4 Example 2 — Working (3/3)

Step 4: Broker and publisher run the 4-step QoS 2 handshake — PUBLISH -> PUBREC (broker confirms receipt) -> PUBREL (publisher releases) -> PUBCOMP (broker confirms completion) — guaranteeing exactly-once delivery to the broker.

Step 5: The broker forwards the alert to the monitoring service with the same QoS 2 handshake; because Retain=true, it also immediately supplies this value to any new subscriber joining later (e.g., a supervisor opening the dashboard mid-shift).

Step 6: If motor 7's sensor stops sending anything for more than $1.5 \times 30s$, the broker automatically publishes its Last Will ("offline", Retained) — alerting the service without any polling.

3.4 Example 2 — QoS 2 + Last Will Sequence Diagram



3.5 MQTT — Advantages (1/2)

Advantages

Publish/subscribe via a broker naturally supports many-to-many, fan-out telemetry at scale.

Three QoS levels let each message trade reliability against overhead as needed.

Retained messages give new subscribers immediate access to the last known state.

3.5 MQTT — Advantages (2/2)

Advantages

Last Will and Testament enables automatic, broker-driven offline/liveness detection.

Runs over TCP, inheriting ordered, reliable delivery and working well through most firewalls (especially on port 443/8883).

Very compact fixed header (~2 bytes) plus variable payload keeps bandwidth use low.

3.5 MQTT — Disadvantages (1/2)

Disadvantages

Requires a broker as a single point of coordination — its availability and scaling become critical.

TCP connection and keep-alive overhead is heavier than CoAP's connectionless UDP model.

QoS 2's four-step handshake adds latency and complexity compared to CoAP's simpler retry model.

3.5 MQTT — Disadvantages (2/2)

Disadvantages

No native request/response semantics; a strict RPC-style interaction needs extra topic conventions.

Broker becomes a potential bottleneck and a single point of failure unless clustered.

Security and access control (topic-level ACLs) must be configured on the broker, not the protocol.

4. CoAP vs MQTT — Quick Comparison

Aspect	CoAP	MQTT
Transport	UDP (connectionless)	TCP (connection-oriented)
Model	Request/Response, + Observe for push	Publish/Subscribe via broker
Reliability	Built into CoAP (CON/ACK, backoff)	TCP + QoS 0/1/2 at app layer
Best fit	Individually-addressed constrained nodes	Many-to-many telemetry, device fleets

5. Summary

- CoAP suits battery-powered, individually addressable constrained nodes needing RESTful request/response interaction over lossy, low-power links; Observe adds lightweight push capability.
- MQTT suits many-to-many telemetry at scale, where a broker-mediated publish/subscribe model, tunable QoS, retained state, and Last-Will notifications let large device fleets exchange data reliably and asynchronously.
- In practice, IoT architectures often use both together: CoAP close to the constrained sensor/actuator nodes, and MQTT for aggregating and distributing that data across the wider system.

Thank You

Questions & Discussion